```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy.special import loggamma

def run_boltzmann_mattress_model():
    print("--- Loading Mattress Sales Mock Data ---")
    df = pd.read_csv("mattress_sales_mockup_3years.csv")
    df['Date'] = pd.to_datetime(df['Date'])

    # Aggregate weekly total sales for the top panel
    weekly_sales = df.groupby('Date')['Units_Sold'].sum().reset_index()
    weekly_sales = weekly_sales.sort_values('Date').reset_index(drop=True)

    # Boltzmann Entropy Function based on Microstate Multiplicity (W)
    # S = k_B * ln(W), where W = N! /prod(ni!) for SKU/Manufacturer distributions
    def compute_boltzmann_entropy(sub_df):
        counts = sub_df['Units_Sold'].values
        N = counts.sum()
        if N == 0:
            return 0.0
        # Use loggamma for Stirling / factorial approximation of ln(W)
        log_W = loggamma(N + 1) - np.sum(loggamma(counts + 1))
        return float(log_W)

    # Compute weekly Boltzmann entropy across manufacturer/SKU configurations
    dates = []
    boltzmann_entropy = []

    for date, group in df.groupby('Date'):
        dates.append(date)
        boltzmann_entropy.append(compute_boltzmann_entropy(group))

    entropy_df = pd.DataFrame({'Date': dates, 'Boltzmann_Entropy': boltzmann_entropy})
    entropy_df = entropy_df.sort_values('Date').reset_index(drop=True)

    # Merge with weekly sales
    merged = pd.merge(weekly_sales, entropy_df, on='Date')
    merged['Rolling_Mean_Sales'] = merged['Units_Sold'].rolling(window=12, min_periods=1
    ).mean()
    merged['Rolling_Mean_Entropy'] = merged['Boltzmann_Entropy'].rolling(window=12,
    min_periods=1).mean()

    # Display console summary
    print(f"Total Weeks Analyzed: {len(merged)}")
    print(f"Latest Weekly Sales Volume: {merged['Units_Sold'].iloc[-1]} units")
    print(f"Latest Boltzmann Entropy (S = ln W): {merged['Boltzmann_Entropy'].iloc[-1]
    :.2f}")
    print("\n--- Q4 Sales Trend Prediction & Combinatorial State Analysis ---")
    print("Interpretation: Boltzmann entropy measures the combinatorial diversity of
    consumer choices across SKUs.")
    print("Higher entropy indicates sales are spread across diverse product
    configurations; lower entropy points to concentrated demand.")

    # Plotting Results
    fig, axes = plt.subplots(2, 1, figsize=(12, 9), sharex=True)

    # Sales Volume Subplot
    axes[0].plot(merged['Date'], merged['Units_Sold'], label='Weekly Units Sold', color=
    '#2b5c8f', alpha=0.5)
    axes[0].plot(merged['Date'], merged['Rolling_Mean_Sales'], label='12-Week Moving
    Average', color='#d95f02', linewidth=2.5)
    axes[0].set_ylabel('Mattress Unit Sales', fontsize=11)
    axes[0].set_title('Mattress Category Sales Trend & Q4 Forecasting Outlook (Boltzmann
    Model)', fontsize=13, fontweight='bold')
    axes[0].legend(loc='upper left')
    axes[0].grid(True, linestyle='--', alpha=0.5)
```

```
60
61      # Boltzmann Entropy Subplot
62      axes[1].plot(merged['Date'], merged['Boltzmann_Entropy'], label='Boltzmann Entropy
        ($S = \\ln W$)', color='#1b9e77', linewidth=1.5, alpha=0.7)
63      axes[1].plot(merged['Date'], merged['Rolling_Mean_Entropy'], label='12-Week Moving
        Average (Entropy)', color='#7570b3', linewidth=2)
64      axes[1].axhline(y=merged['Boltzmann_Entropy'].mean(), color='red', linestyle=':',
        label='Mean Entropy Baseline')
65      axes[1].set_ylabel('Boltzmann Entropy ($S$)', fontsize=11)
66      axes[1].set_xlabel('Date', fontsize=11)
67      axes[1].set_title('Combinatorial Configuration Entropy (Statistical Mechanics
        Approach)', fontsize=13, fontweight='bold')
68      axes[1].legend(loc='upper left')
69      axes[1].grid(True, linestyle='--', alpha=0.5)
70
71      plt.tight_layout()
72      plt.savefig("mattress_boltzmann_forecast.png")
73      print("Chart successfully saved to 'mattress_boltzmann_forecast.png'.")
74
75  if __name__ == '__main__':
76      run_boltzmann_mattress_model()
```